

6-Week Frontend Sprint Plan

Task breakdown and timeline for 2 junior frontend developers working full-time (8 hrs/day, 5 days/week) on the fox-passport-republic-app.

- 2 JUNIOR DEVS
- 6 WEEKS · 40 HRS/WK
- NEXT.JS FRONTEND
- 4 PRIORITY FEATURES
- LEAD OWNS BE

SPRINT SUMMARY

WEEK	DEV A FOCUS	DEV B FOCUS	KEY DELIVERABLE
Week 1	Onboarding + Passport API Layer	Onboarding + Match Flow Completion	<code>lib/api/passport.ts</code> + Match confirmation view
Week 2	Passport Full Wiring + Citizen Dashboard	Notification System	Passport page live · Notification bell + dropdown
Week 3	Host Dashboard Audit + Polish	Mayor Dashboard Audit + Polish	All creator dashboards API-complete
Week 4	Foxer Dashboard + User Settings	Search + Venue Discovery Polish	All citizen-facing discovery pages polished
Week 5	Mobile Audit + Citizen E2E Test Loop	Mobile Audit + Foxer E2E Test Loop	Mobile-responsive · Bug list documented
Week 6	Bug Fixes + Playwright Tests	Bug Fixes + Playwright Tests	3 E2E tests each · Sprint review + handover

⚠️ HARD DEPENDENCIES — RESOLVE BEFORE SPRINT STARTS

- ! Passport BE endpoints must be live by start of Week 2. Dev A spends Week 1 writing API stubs and type mappers so they can drop in real calls the moment endpoints go live. If BE is late, Dev A loses a full week.
- ! GET /api/v1/notifications must exist before Day 6 (Monday of Week 2). Dev B's entire Week 2 plan depends on this endpoint. Coordinate with Lead before the sprint starts.
- ! GET /api/v1/passport/leaderboard must be scoped. If building a global XP leaderboard is non-trivial on the BE, defer the Global Ranks tab to Week 3 and have Dev A stub a loading placeholder.

BOTH DEVS

Day 1 · Mon

Codebase Onboarding

Both devs spend the first day understanding the project structure, running the app locally, and reading the domain glossary before touching any code.

- Clone repo, run `pnpm install` and `pnpm dev` — confirm app loads
- Read `CONTEXT.md` in full — understand all 5 roles (Citizen, Fox, Mayor, Host, Admin) and how they relate
- Dev A traces the booking flow: `/booking → /checkout → /booking/fulfillment`
- Dev B traces the match flow: `/venues → /match/[foxerId] → Passport matches tab`
- Both read through `lib/api/` folder to understand the existing API call pattern before writing new ones

DEV A

Days 2-5 · Tue-Fri

Passport API Service Layer

All data in `PassportClient.tsx` is hardcoded mock data (comment reads "will be replaced with actual API calls"). This week, Dev A builds the service layer and starts wiring the Stamps tab.

- Day 2: Create `lib/api/passport.ts` — stub functions following the pattern of `lib/api/bookings.ts`
 - `getMyPassport()` → GET `/api/v1/passport` (XP paths + summary)
 - `getMyStamps()` → GET `/api/v1/passport/stamps`
 - `getMyBadges()` → GET `/api/v1/passport/badges`
 - `getGlobalRanks()` → GET `/api/v1/passport/leaderboard`
 - `markStampSeen(id)` → PATCH `/api/v1/passport/stamps/:id/seen`
- Day 3: Read `types/gamification.ts` carefully. Write mapper functions in `lib/mappers/passport.ts` that convert raw API responses to `PathProgress[]`, `Badge[]`, and `PassportStamp[]` types
- Day 4: In `PassportClient.tsx`, replace inline `mockPaths` array with `useEffect` + `getMyPassport()` call. Add loading + empty states for Progress tab
- Day 5: Replace `MOCK_PASSPORT_STAMPS` import from `lib/gamification-data.ts` with `getMyStamps()` call. Wire the existing empty state — it just needs the condition to be `stamps.length === 0`

DEV B

Days 2-5 · Tue-Fri

Match Flow Completion

`MatchConfig.tsx` handles creating a match request (3-step wizard). It fetches real Foxers from the API and calls `createMatch()`. What's missing: the post-submission confirmation view and error handling.

- Day 2: Read `MatchConfig.tsx` in full. Map all 3 steps (style → date/guests → review). Confirm `foxerId` from `useParams()` correctly pre-selects the Foxer from the fetched list — fix if not
- Day 3: Build step 4 (post-submission confirmation) — shown after `createMatch()` resolves: Foxer avatar + name, match status badge "Under Review", estimated response time, link to Passport Matches tab
- Day 4: Handle all error cases — API failure (toast + retry), Foxer unavailable, missing fields. Add loading spinner on submit button during `isSubmitting` (state already exists)
- Day 5: E2E test the full flow. Confirm newly created match appears via `getMyMatches()` in Passport matches tab (align both features for Week 2 wiring)

DEV A

Days 6-10 · Mon-Fri

Passport Full Wiring + Citizen Dashboard

Assumes BE Passport endpoints are live by Monday Week 2. Dev A completes all three Passport tabs then pivots to the citizen user dashboard.

- Day 6: Replace `PassportClient.tsx` with real `getMyBadges()` call. Feed owned badge IDs into the existing `ownedIds / finalLockedIds` logic. Add loading skeleton for badge grid
- Day 7: Wire the Matches tab — replace hardcoded "Jasmine L." card with real `getMyMatches()` data. Map match status field to progress bar stages: pending → Sent, accepted → Reviewing, matched → Matched, completed → Secured
- Day 8: Replace hardcoded global ranks table with real `getGlobalRanks()` data. Handle empty/loading state. Raise PR for all Passport work
- Day 9: Read `UserDashboardClient.tsx` — identify which citizen dashboard props are hardcoded vs. fetched. Create/extend `lib/api/profile.ts` to fetch wallet balance and recent transactions
- Day 10: Wire `UserWallet` with real wallet data. Wire `UserNextUp` with upcoming bookings from existing bookings API. Raise PR

DEV B

Days 6-10 · Mon-Fri

Notification System (Full Build)

No notification system exists anywhere in the frontend — no component, no store, no hook. This is a full build from scratch. Scope to polling (not socket.io) to keep it junior-safe. Requires BE endpoint to be ready.

- Day 6: Confirm with Lead that GET `/api/v1/notifications` endpoint exists. Create `lib/api/notifications.ts` with `getNotifications()`, `markAsRead(id)`, `markAllRead()`
- Day 7: Build `components/shared/NotificationBell.tsx` — bell icon with red unread badge count. Wire to `Navbar.tsx` next to user avatar
- Day 8: Build `components/shared/NotificationDropdown.tsx` — slides open below bell. Lists 10 most recent notifications with icon (by type), message text, and relative timestamp ("2 min ago"). "Mark all as read" button at top
- Day 9: Create `store/useNotificationStore.ts` to hold notification count globally — so badge number updates across the whole app without each component re-fetching
- Day 10: Add polling — fetch `getNotifications()` every 30s via `useEffect + setInterval`. Optimistically mark as read when dropdown opens. Test: book something, confirm notification appears in bell within 30s. Raise PR

DEV A

Days 11-15 · Mon-Fri

Host Dashboard Audit + Polish

The host dashboard (`/host`) has pages for venues, assets, services, events, and calendar. Components exist in `components/host/`. Each needs to be audited for API wiring completeness and visual consistency.

- Day 11-12: Audit `HostVenuesClient.tsx` and `HostEventsClient.tsx`. Confirm venue/event lists and status badges (draft/pending/published) read from real API. Fix any hardcoded data
- Day 13: Audit `HostCalendarClient.tsx` — most complex page. Confirm booked dates render from real booking data. No placeholder dates
- Day 14: Audit `HostAssetsClient.tsx` and `HostServicesClient.tsx`. These are listing management pages — wire any missing data
- Day 15: Cross-page test: create venue → submit for review → check it appears in host dashboard with `pending_review` status. Fix any data-flow gaps. Raise PR

DEV B

Days 11-15 · Mon-Fri

Mayor Dashboard Audit + Polish

The mayor dashboard (`/mayor`) is the space provider role. Their primary flow: list venues → get admin approval → venues appear in discovery. Confirm this entire chain works end-to-end.

- Day 11-12: Audit Mayor venues page (`/mayor/venues`). Confirm created venues appear with correct status and edit/delete actions. Check `/mayor/create-venue` form submits correctly to `POST /api/v1/venues`
- Day 13: Audit `/mayor/apply` (role application form). Confirm it submits a role request to `POST /api/v1/role-requests` and handles all document upload fields (NBI, TIN, valid IDs)
- Day 14: Audit Mayor events and assets pages for missing API connections
- Day 15: Cross-page test: list a venue → get approved via admin dashboard → confirm it shows up in `/venues` discovery. Fix any chain gaps. Raise PR

DEV A

Days 16-20 • Mon-Fri

Foxer Dashboard + User Settings

The Foxer (creator) dashboard covers listing management, earnings, and earnings history. FoxerEarningsClient.tsx is already wired — it just needs a visual QA pass. Main work is the listing-builder forms and user settings page.

- Day 16-17: Audit components/foxer/listing-builder/ and components/foxer/event-builder/ . Confirm asset/service listing creation calls POST /api/v1/asset and POST /api/v1/service. Confirm created listings appear in Foxer dashboard
- Day 18: Visual QA pass on FoxerEarningsClient.tsx — this is the most complete component already. Check escrow/payout states render correctly with real booking data
- Day 19-20: Audit /user/settings ProfileSettingsClient.tsx). Confirm PATCH /api/v1/profile call works. Confirm updated name/photo reflects immediately in nav avatar and Passport sidebar. Raise PR

DEV B

Days 16-20 • Mon-Fri

Search + Venue Discovery Polish

The /search and /venues pages are the primary citizen-facing discovery surface. High visibility. Confirm filters, results, venue detail, and "For You" recommendations all work with real data.

- Day 16-17: Audit /search page. Confirm category, location, and price range filters pass correctly as query params to the API. Confirm results render with correct venue cards and empty states
- Day 18-19: Audit /venues discovery page. Confirm venue cards link to correct detail pages. Check /venues/[id] loads all venue data (gallery, capacity, base rate, assets, services) from real API calls
- Day 20: Check UserForYou.tsx in the citizen dashboard — if hardcoded, wire it to GET /api/v1/event-templates with a category filter. Raise PR

BOTH DEVS

Days 21-23 • Mon-Wed

Mobile Audit (390px viewport)

Go through every page at iPhone 15 size (390px). Flag overflow, broken layouts, unreadable text, and untappable buttons. Fix each issue as it's found — these are typically small CSS changes.

- Priority pages: Home, Venue Detail, Booking flow, Passport, Match Config, Notification dropdown, Admin dashboard
- Fix overflow issues — add sm: breakpoints, fix flex wrapping, correct font sizes
- Confirm all buttons have minimum 44px tap target height
- Confirm modal/dialog components close correctly on mobile tap-outside
- Test Passport page layout on mobile — the sidebar may need to collapse to a bottom nav or drawer on small screens

DEV A

Days 24-25 • Thu-Fri

Citizen E2E Integration Test Loop

Manually test the complete Citizen journey from account creation through to Passport stamp. Document every bug found with screen recording or screenshot.

- Register as a new Citizen → complete onboarding
- Discover a venue via /search → navigate to venue detail
- Book an event → go through full checkout → confirm payment
- Navigate to /booking/fulfillment → confirm arrival
- Check /user/passport → confirm stamp appears in Stamps tab after booking completes
- Document all bugs: label as Blocker / Polish / Defer

DEV B

Days 24-25 • Thu-Fri

Foxer E2E Integration Test Loop

Manually test the complete Foxer journey from application through to payout. Document every bug found.

- Apply as a Foxer (foxerAsset or foxerService) → confirm role request submitted
- Approve the role request via Admin dashboard → confirm Foxer dashboard unlocks
- Create an Asset or Service listing → confirm it appears in listings page
- Receive a booking (from a Citizen test account) → go to /creator-dashboard/earnings
- Confirm escrow state shows "In Escrow" → Citizen confirms arrival → verify "Paid Out" state
- Document all bugs: label as Blocker / Polish / Defer

BOTH DEVS

Days 26–28 · Mon–Wed

Bug Fixes from Week 5 Triage

Work through the bug list produced in Week 5. Focus on Blockers first, then Polish items. Each fix gets its own small PR — no bundled fixes, easier for Lead to review.

- Blocker bugs fixed by Wednesday at the latest — these prevent core flows from working
- Polish bugs addressed with small focused PRs — one bug per PR, clear description
- Deferred bugs written as GitHub issues so nothing is lost after sprint ends
- Each PR must include: what was broken, what was changed, how to test it

DEV A

Day 29 · Thu

Playwright Tests — Citizen Flows

The repo already has playwright.config.ts. Write 3 E2E tests covering the Citizen's critical paths.

- **Test 1:** Citizen can log in and the Passport page loads with correct user name in the sidebar
- **Test 2:** Booking flow completes — fills checkout form → Stripe test payment → redirects to success page with booking confirmation
- **Test 3:** Passport Stamps tab shows the correct stamp after a completed booking (requires test data setup)

DEV B

Day 29 · Thu

Playwright Tests — Foxer + Notification Flows

Write 3 E2E tests covering the Foxer's critical path and the Notification system.

- **Test 1:** Match request can be created from a Foxer profile page — wizard completes, confirmation step renders with correct Foxer name
- **Test 2:** Notification bell shows correct unread count after a booking is confirmed; clicking it opens the dropdown with the notification listed
- **Test 3:** Search page returns filtered results when a category is selected — result count is greater than zero for seeded test data

BOTH DEVS + LEAD

Day 30 · Fri

Sprint Review + Handover

Final day — walk through every page with the Lead Developer. Document what's done, what's partial, and what's deferred. Any deferred items become GitHub issues so nothing is lost.

- Live demo of all 4 features: Passport, Match flow, Notifications, all dashboards
- Review Playwright test results — all 6 tests must pass before sign-off
- Create GitHub issues for any deferred items — label as enhancement or bug with priority
- Brief knowledge transfer — Dev A documents any non-obvious decisions made in Passport wiring; Dev B documents the Notification polling architecture